310-2202 โครงสร้างของระบบคอมพิวเตอร์
(Computer Organization)

Topic 7: Assembly Language Program

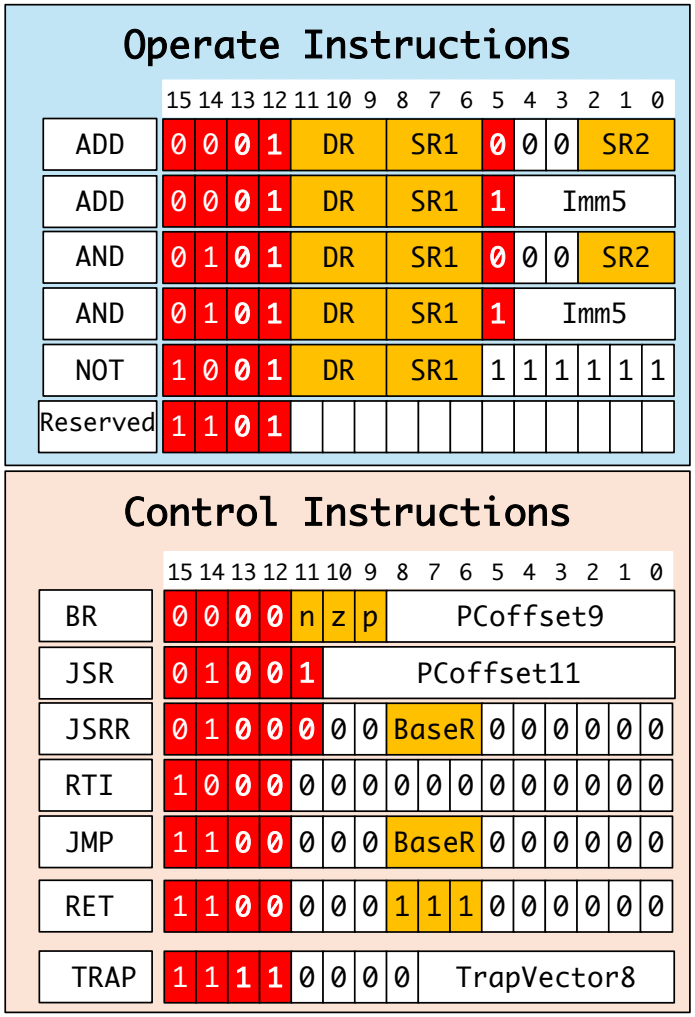
Damrongrit Setsirichok

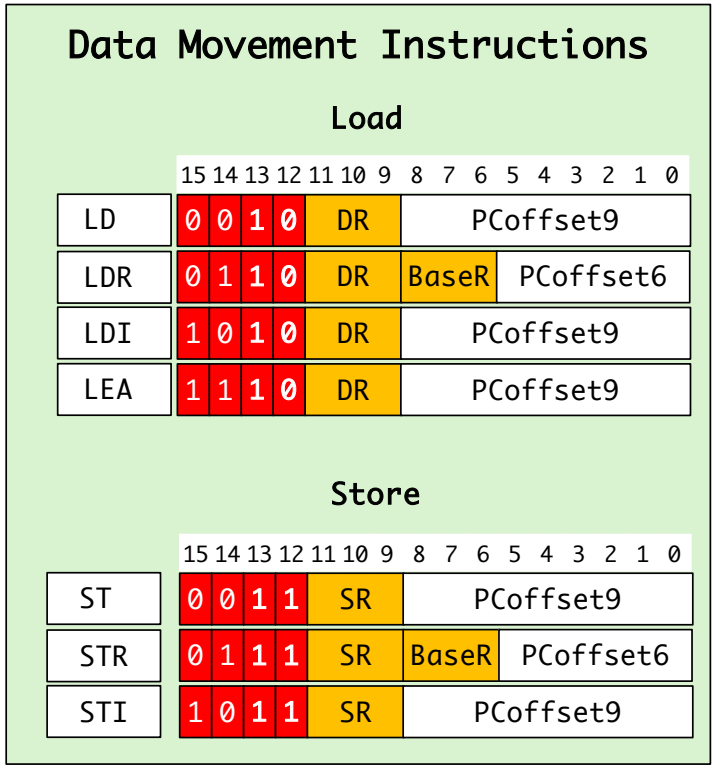
Topic

- Assembly Language Overview
- The assembly Process

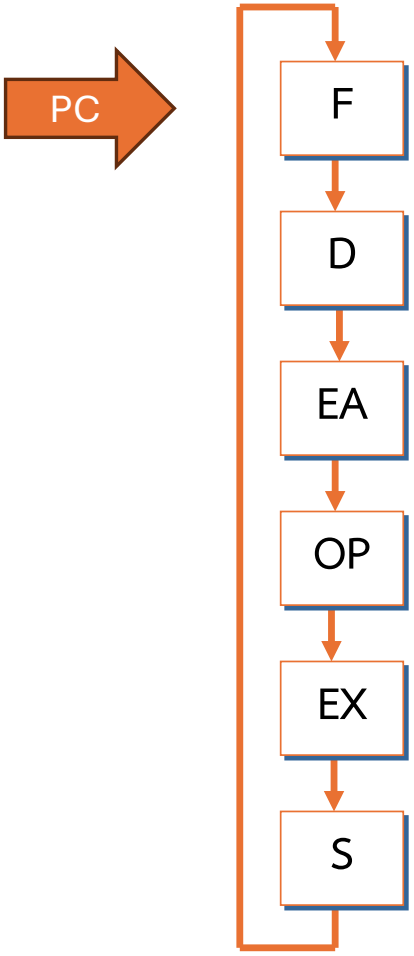
Assembly Language Overview

LC-3 ISA Group





Instruction Processing: Finite State Automata



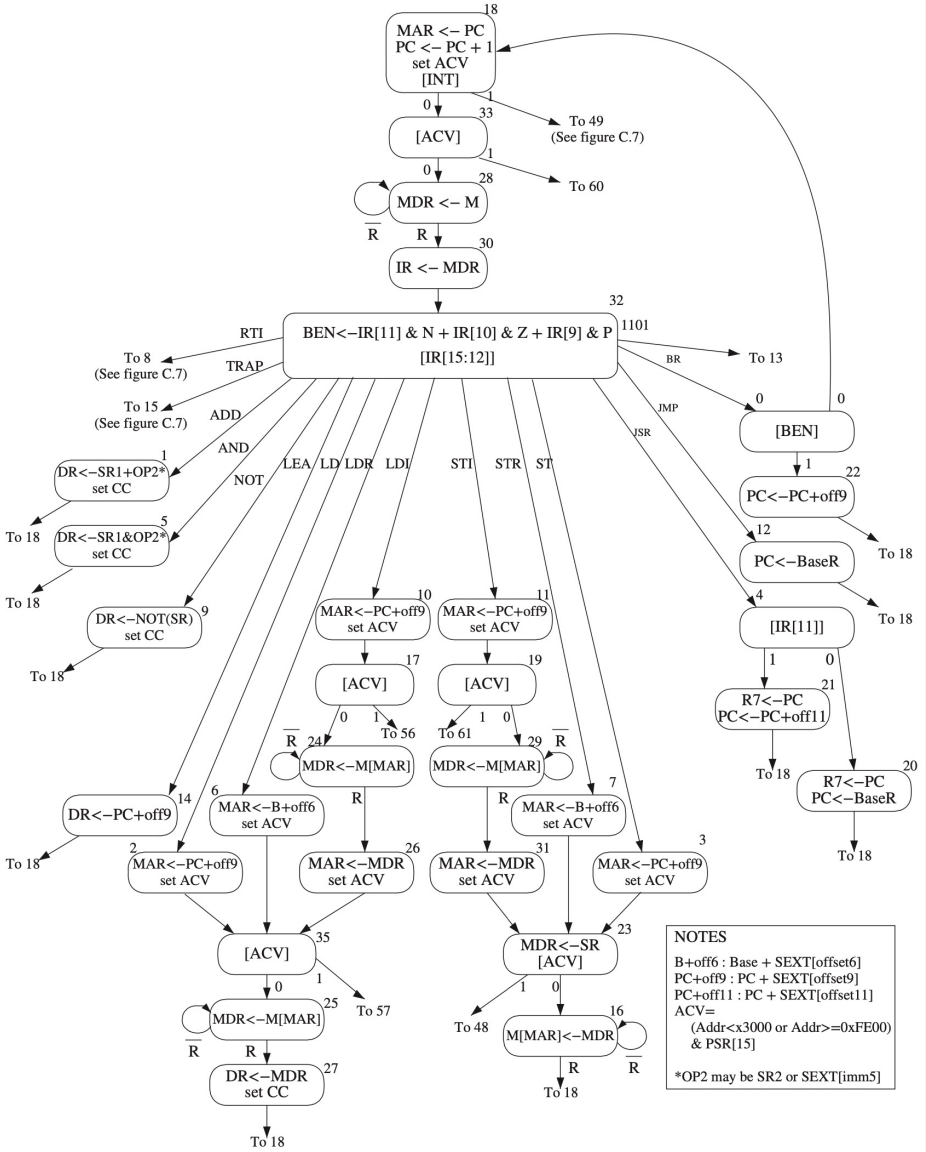
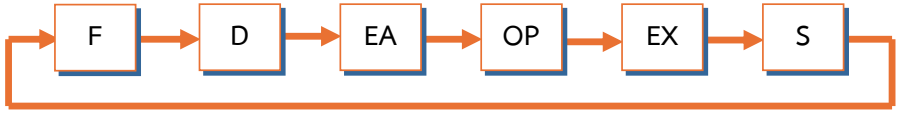
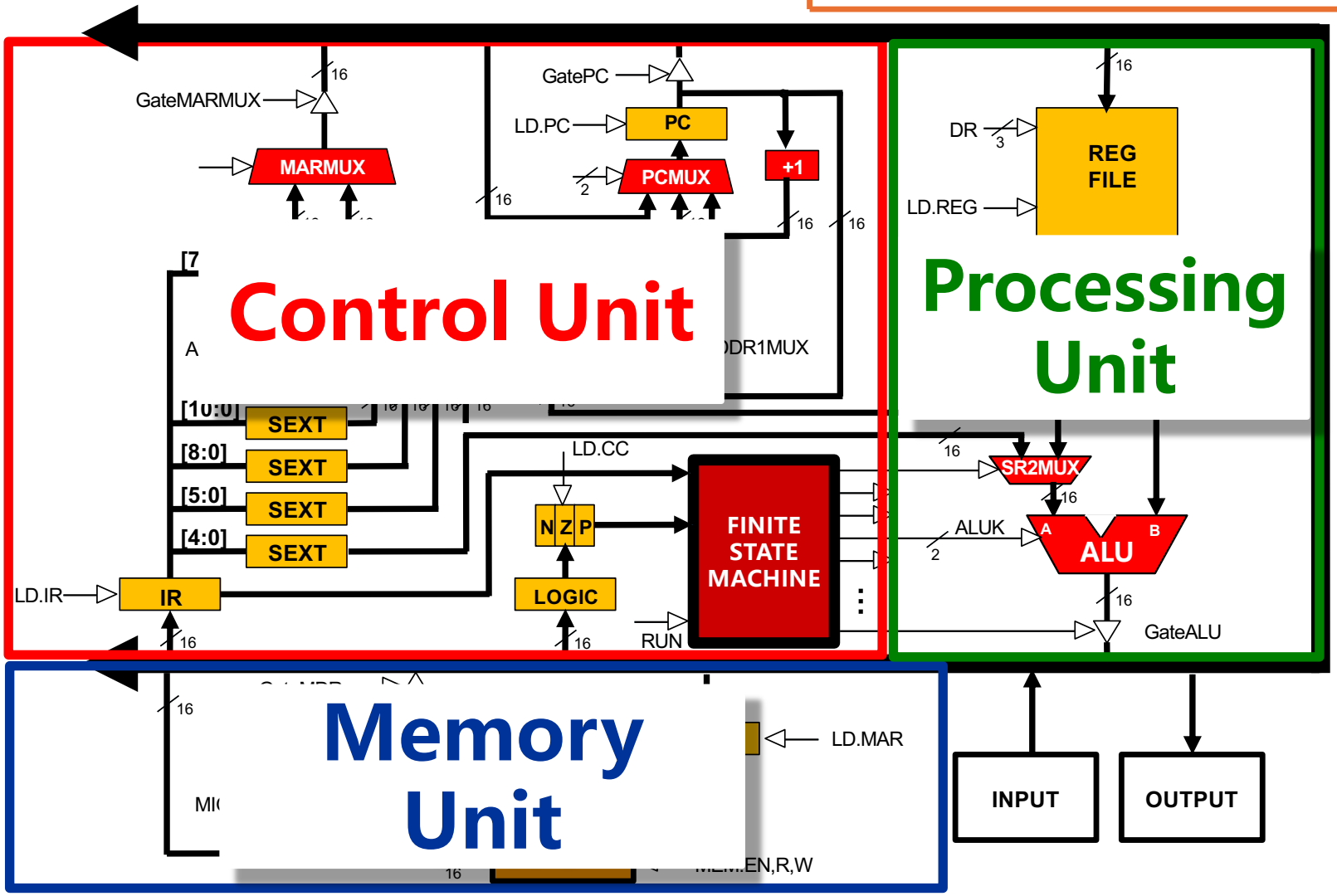


Figure C.2 A state machine for the LC-3.

Recall LC-3 Data Path (von Neumann Model)





An LC-3 Program

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	R3 <- M[x3012]
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	TRAP x23
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	R4 <- R1-4
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	BRz x300E
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	R1 <- NOT R1
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1 + 1
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	R1 <- R1 + R0
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	BRnp x300B
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	R2 <- R2 + 1
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	R3 <- R3 + 1
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	BRnzp x3004
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	R0 <- M[x3013]
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	R0 <- R0 + R2
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	TRAP x21
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	TRAP x25
x3012	Starting address of file																
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	ASCII TEMPLATE

0000	BR
0001	ADD
0010	LD
0101	AND
1111	TRAP

Human-Readable Machine Language

- Computers like ones and zeros…

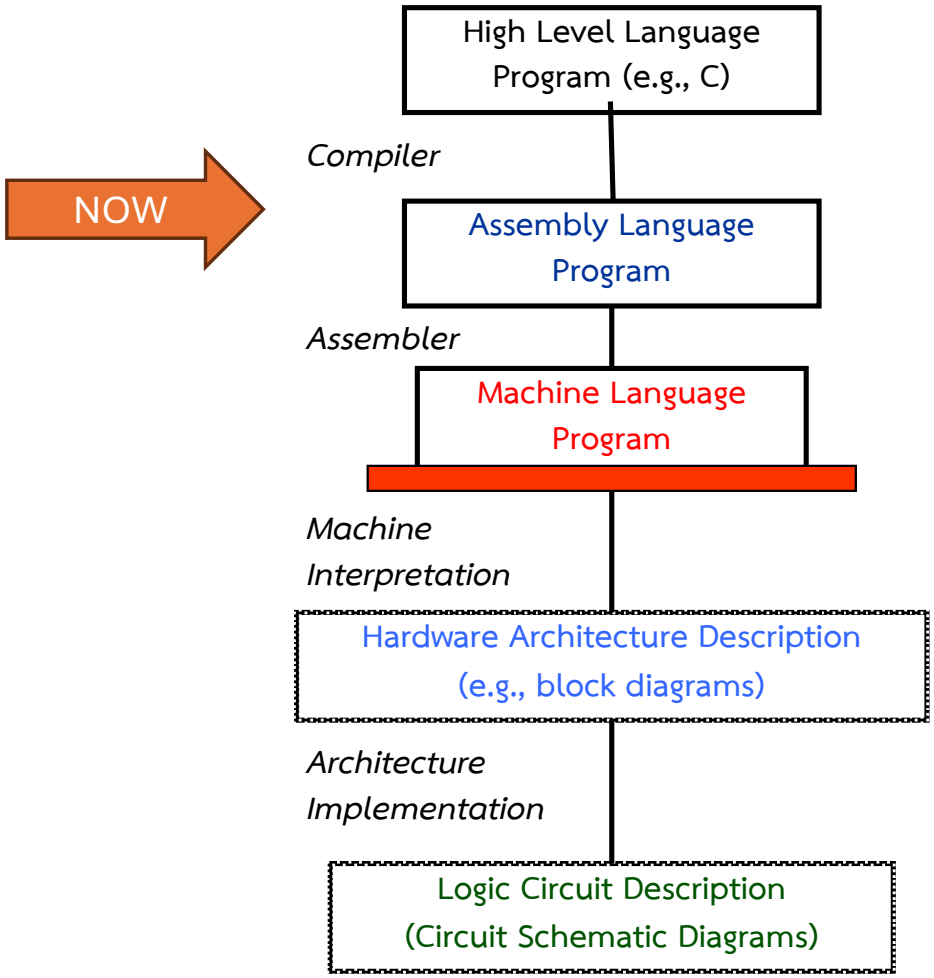
  0001110010000110

- Humans like symbols… **ADD R6, R2, R6**

  C = a + b;

- **Assembler is a program that turns symbols into machine instructions.**
  - ISA-specific: close correspondence between symbols and instruction set
    - mnemonics for opcodes
    - labels for memory locations
  - Additional operations for allocating storage and initializing data

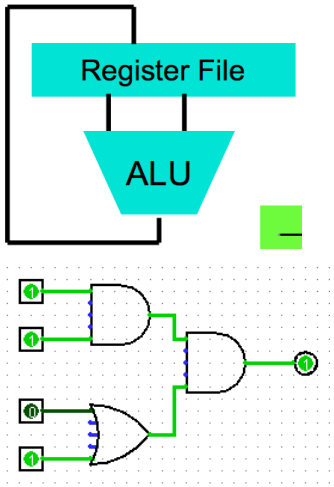
Software and Hardware Co-design




```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

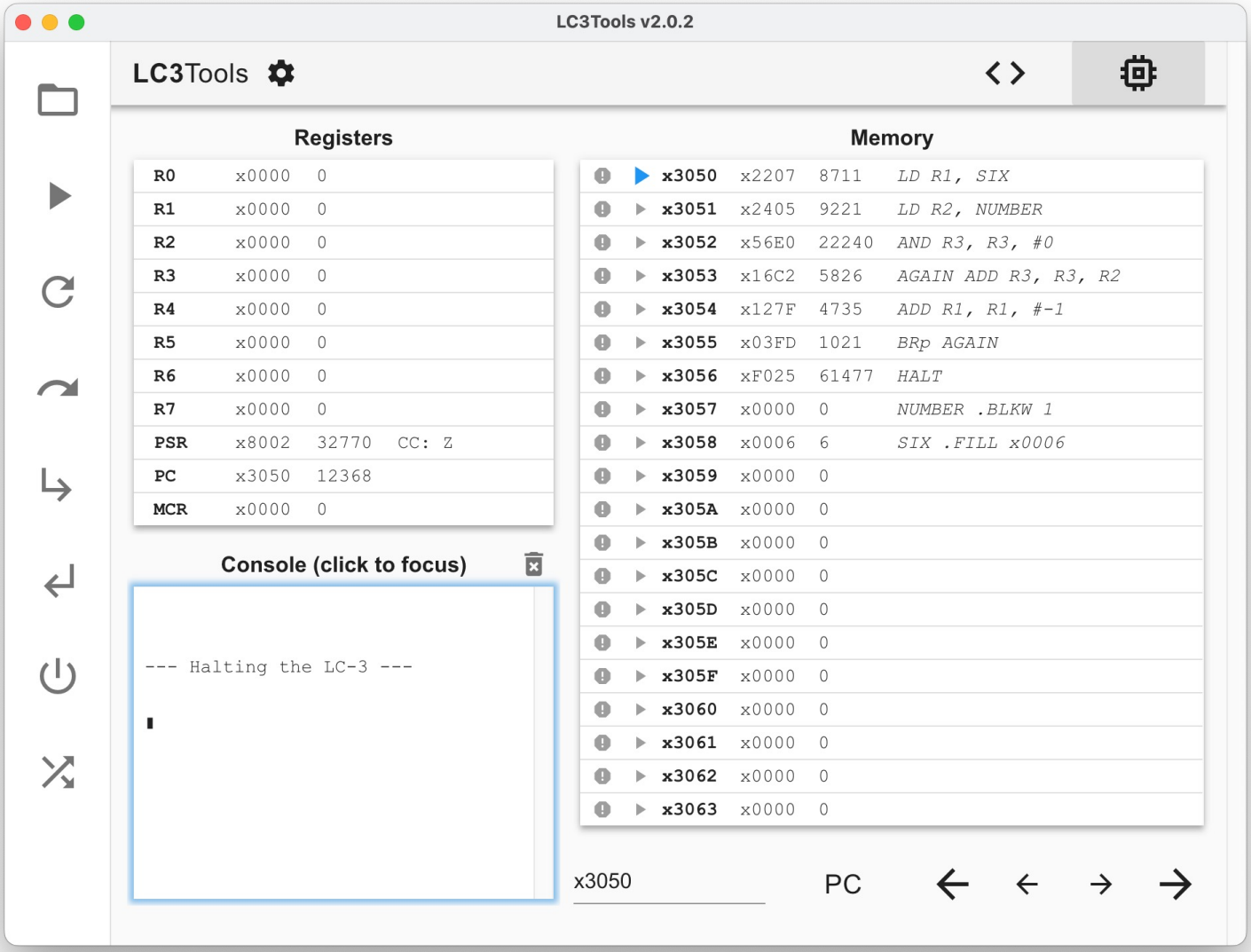
```
lw  $t0, 0(a0)
lw  $t1, 4(a0)
sw  $t1, 0(a0)
sw  $t0, 4(a0)
```

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```



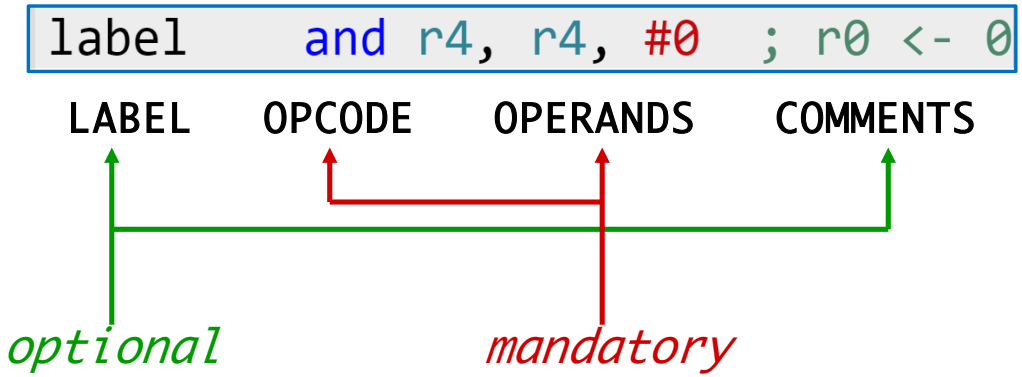
Assambly Program

```
;
; Program to multiply a number by the constant 6
;
        .ORIG  x3050
        LD     R1, SIX
        LD     R2, NUMBER
        AND    R3, R3, #0     ; Clear R3.  It will
                              ; contain the product.
; The inner loop
;
AGAIN   ADD    R3, R3, R2
        ADD    R1, R1, #-1    ; R1 keeps track of
        BRp    AGAIN          ; the iteration.
;
        HALT
;
NUMBER  .BLKW  1
SIX     .FILL  x0006
;
        .END
```


LC3Tools v2.0.2
LC3Tools
Registers
R0 x0000 0
R1 x0000 0
R2 x0000 0
R3 x0000 0
R4 x0000 0
R5 x0000 0
R6 x0000 0
R7 x0000 0
PSR x8002 32770 CC: Z
PC x3050 12368
MCR x0000 0
Console (click to focus)
--- Halting the LC-3 ---
Memory
x3050 x2207 8711 LD R1, SIX
x3051 x2405 9221 LD R2, NUMBER
x3052 x56E0 22240 AND R3, R3, #0
x3053 x16C2 5826 AGAIN ADD R3, R3, R2
x3054 x127F 4735 ADD R1, R1, #-1
x3055 x03FD 1021 BRp AGAIN
x3056 xF025 61477 HALT
x3057 x0000 0 NUMBER .BLKW 1
x3058 x0006 6 SIX .FILL x0006
x3059 x0000 0
x305A x0000 0
x305B x0000 0
x305C x0000 0
x305D x0000 0
x305E x0000 0
x305F x0000 0
x3060 x0000 0
x3061 x0000 0
x3062 x0000 0
x3063 x0000 0
x3050
PC

LC-3 Assembly Language Syntax

- Each line of a program is one of the Instruction following:
  - Opcodes and Operands
  - Assembler Directives (or Pseudo-Ops)
  - Comments
  - Empty



- **Whitespace** (between symbols) and case are **ignored**
- Comments (beginning with ";") are also ignored
- An instruction has the following format:

Opcodes and Operands

- Opcodes
  - **Reserved symbols** that correspond to LC-3 instructions
  - listed in Appendix A
    - ex: ADD, AND, LD, LDR, …
- Operands
  - **Registers** -- specified by Rn, where n is the register number
  - **Numbers** -- indicated by # (decimal) or x (hex)
  - **Label** -- symbolic name of memory location
  - **Separated by comma**
  - **Number**, **order**, and **type** correspond to instruction format

```
ADD     R1, R1, R3
ADD     R1, R1, #3
LD      R6, NUMBER
BRz     LOOP
```

Labels and Comments

- **Label**
  - Placed at the **beginning of the line**
  - Assigns a symbolic name to the **address** corresponding to line

    Ex:

```
LOOP ADD R1, R1, #-1
     BRp LOOP
```

- **Comment**
  - Anything after a **semicolon** (“;”) is a comment
  - Ignored by assembler
  - Used by humans to document/understand programs

Assembler Directives

- Pseudo-Operations
  - Do not refer to operations executed by program
  - Used by assembler
  - Look like instruction, but “opcode” starts with dot

Opcode	*Operand*	*Meaning*
.ORIG	address	starting address of program
.END		end of program (does not stop the program)
.BLKW	n	allocate n words of storage (the value not yet known)
.FILL	n	allocate one word, initialize with value n
.STRINGZ	n-character string	allocate n+1 locations, initialize w/characters and null terminator

Example

```
        .ORIG X3010
HELLO .STRINGZ “Hello, World!”
```

00	nul	10	dle	20	sp	30	0	40	@	50	P	60	`	70	p
01	soh	11	dc1	21	!	31	1	41	A	51	Q	61	a	71	q
02	stx	12	dc2	22	"	32	2	42	B	52	R	62	b	72	r
03	etx	13	dc3	23	#	33	3	43	C	53	S	63	c	73	s
04	eot	14	dc4	24	$	34	4	44	D	54	T	64	d	74	t
05	enq	15	nak	25	%	35	5	45	E	55	U	65	e	75	u
06	ack	16	syn	26	&	36	6	46	F	56	V	66	f	76	v
07	bel	17	etb	27	'	37	7	47	G	57	W	67	g	77	w
08	bs	18	can	28	(	38	8	48	H	58	X	68	h	78	x
09	ht	19	em	29	)	39	9	49	I	59	Y	69	i	79	y
0a	nl	1a	sub	2a	*	3a	:	4a	J	5a	Z	6a	j	7a	z
0b	vt	1b	esc	2b	+	3b	;	4b	K	5b	[	6b	k	7b	{
0c	np	1c	fs	2c	,	3c	<	4c	L	5c	\	6c	l	7c	|
0d	cr	1d	gs	2d	-	3d	=	4d	M	5d	]	6d	m	7d	}
0e	so	1e	rs	2e	.	3e	>	4e	N	5e	^	6e	n	7e	~
0f	si	1f	us	2f	/	3f	?	4f	O	5f	_	6f	o	7f	del

```
x3010: x0048
x3011: x0065
x3012: x006C
x3013: x006C
x3014: x006F
x3015: x002C
x3016: x0020
x3017: x0057
x3018: x006F
x3019: x0072
x301A: x006C
x301B: x0064
x301C: x0021
x301D: x0000
```

Trap Codes

- LC-3 assembler provides "pseudo-instructions" for each trap code

  (don't have to remember)

Code	*Equivalent*	*Description*
HALT	TRAP x25	Halt execution and print message to console.
IN	TRAP x23	Print prompt on console, read (and echo) one character from keyboard. Character stored in R0[7:0].
OUT	TRAP x21	Write one character (in R0[7:0]) to console.
GETC	TRAP x20	Read one character from keyboard. Character stored in R0[7:0].
PUTS	TRAP x22	Write null-terminated string to console. Address of string is in R0.

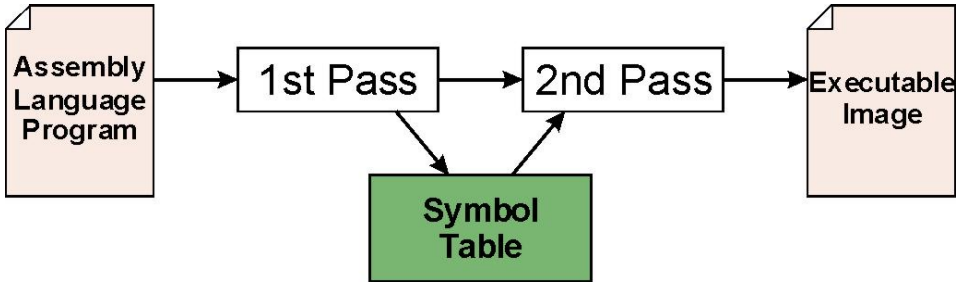
Example:
Count occurrences of a character Revisited

```
01   ;
02   ; Program to count occurrences of a character in a file.
03   ; Character to be input from the keyboard.
04   ; Result to be displayed on the monitor.
05   ; Program works only if no more than 9 occurrences are found.
06   ;
07   ;
08   ; Initialization
09   ;
0A           .ORIG   x3000
0B           AND     R2,R2,#0       ; R2 is counter, initialize to 0
0C           LD      R3,PTR         ; R3 is pointer to characters
0D           TRAP    x23            ; R0 gets character input
0E           LDR     R1,R3,#0       ; R1 gets the next character
0F   ;
10   ; Test character for end of file
11   ;
12   ;
13   TEST    ADD     R4,R1,#-4      ; Test for EOT
14           BRz     OUTPUT         ; If done, prepare the output
15   ;
16   ; Test character for match.  If a match, increment count.
17   ;
18           NOT     R1,R1
19           ADD     R1,R1,#1       ; R1 <-- -R1
1A           ADD     R1,R1,R0       ; R1 <-- R0-R1. If R1=0, a match!
1B           BRnp    GETCHAR        ; If no match, do not increment
1C           ADD     R2,R2,#1
1D   ;
1E   ; Get next character from the file
1F   ;
20   GETCHAR ADD     R3,R3,#1       ; Increment the pointer
21           LDR     R1,R3,#0       ; R1 gets the next character to test
22           BRnzp   TEST
23   ;
24   ; Output the count.
25   ;
26   OUTPUT  LD      R0,ASCII       ; Load the ASCII template
27           ADD     R0,R0,R2       ; Convert binary to ASCII
28           TRAP    x21            ; ASCII code in R0 is displayed
29           TRAP    x25            ; Halt machine
2A   ;
2B   ; Storage for pointer and ASCII template
2C   ;
2D   ASCII   .FILL   x0030
2E   PTR     .FILL   x4000
2F           .END
```

The Assembly Process

Assembly Process

- Convert assembly language file (.asm) into an executable file (.obj) for the LC-3 simulator



- First Pass:
  - Scan program file
  - Find all labels and calculate the corresponding addresses; this is called the *symbol table*
- Second Pass:
  - Convert instructions to machine language, using information from symbol table

First Pass: Constructing the Symbol Table

1. Find the .ORIG statement, the address of the first instruction.
   - Initialize location counter (LC), which keeps track of the current instruction.

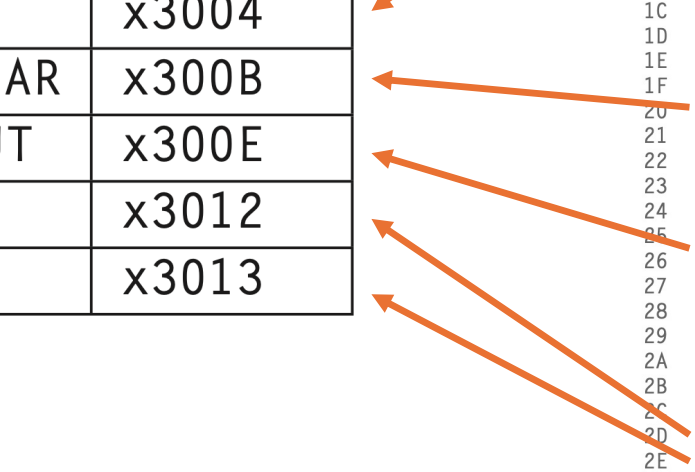
2. For each non-empty line in the program:
   a) If line contains a label, add label and LC to symbol table.
   b) Increment LC.
      - NOTE: If statement is .BLKW or .STRINGZ,
        increment LC by the number of words allocated

3. Stop when .END statement is reached.

*** A line that contains only a comment is considered an empty line.

Example symbol table

Symbol	Address
TEST	x3004
GETCHAR	x300B
OUTPUT	x300E
ASCII	x3012
PTR	x3013



```
01  ;
02  ; Program to count occurrences of a character in a file.
03  ; Character to be input from the keyboard.
04  ; Result to be displayed on the monitor.
05  ; Program works only if no more than 9 occurrences are found.
06  ;
07  ;
08  ; Initialization
09  ;
0A          .ORIG   x3000
0B          AND     R2,R2,#0        ; R2 is counter, initialize to 0
0C          LD      R3,PTR          ; R3 is pointer to characters
0D          TRAP    x23             ; R0 gets character input
0E          LDR     R1,R3,#0        ; R1 gets the next character
0F  ;
10  ; Test character for end of file
11  ;
12  ;
13  TEST    ADD     R4,R1,#-4       ; Test for EOT
14          BRz     OUTPUT          ; If done, prepare the output
15  ;
16  ; Test character for match.  If a match, increment count.
17  ;
18          NOT     R1,R1
19          ADD     R1,R1,#1        ; R1 <-- -R1
1A          ADD     R1,R1,R0        ; R1 <-- R0-R1. If R1=0, a match!
1B          BRnp    GETCHAR         ; If no match, do not increment
1C          ADD     R2,R2,#1
1D  ;
1E  ; Get next character from the file
1F  ;
20  GETCHAR ADD     R3,R3,#1        ; Increment the pointer
21          LDR     R1,R3,#0        ; R1 gets the next character to test
22          BRnzp   TEST
23  ;
24  ; Output the count.
25  ;
26  OUTPUT  LD      R0,ASCII        ; Load the ASCII template
27          ADD     R0,R0,R2        ; Convert binary to ASCII
28          TRAP    x21             ; ASCII code in R0 is displayed
29          TRAP    x25             ; Halt machine
2A  ;
2B  ; Storage for pointer and ASCII template
2C  ;
2D  ASCII   .FILL   x0030
2E  PTR     .FILL   x4000
2F          .END
```

Second Pass: Generating Machine Language

- For each executable assembly language statement, generate the corresponding machine language instruction
  - If operand is a label, look up the address from the symbol table
- Potential problems:
  - Improper number or type of arguments

    ```
    Ex:     NOT     R1, #7
            ADD     R1, R2
            ADD     R3, R3, NUMBER
    ```

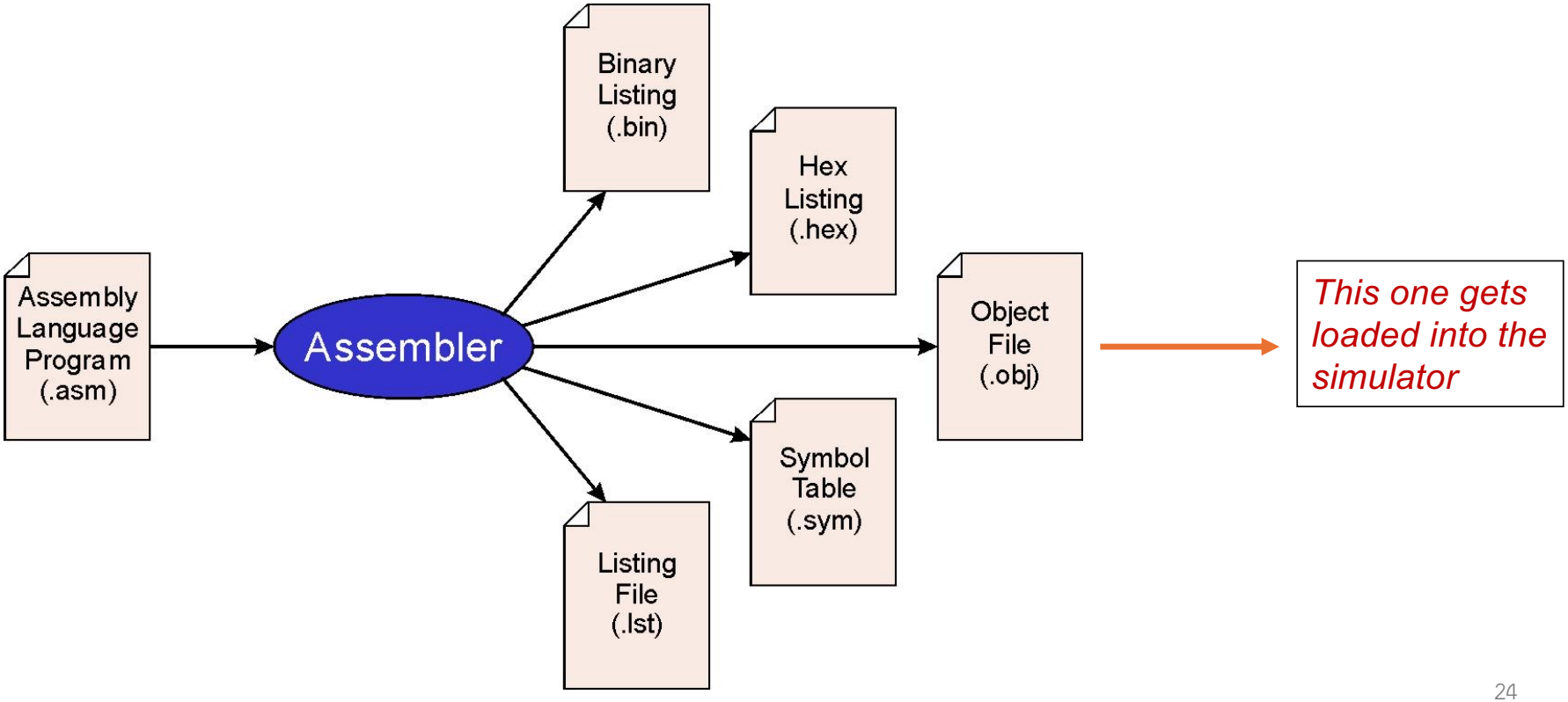
  - Immediate argument too large

    ```
    Ex:     ADD     R1, R2, #1023
    ```

  - Address (associated with label) not on the same page (not within the range +256 or −255)

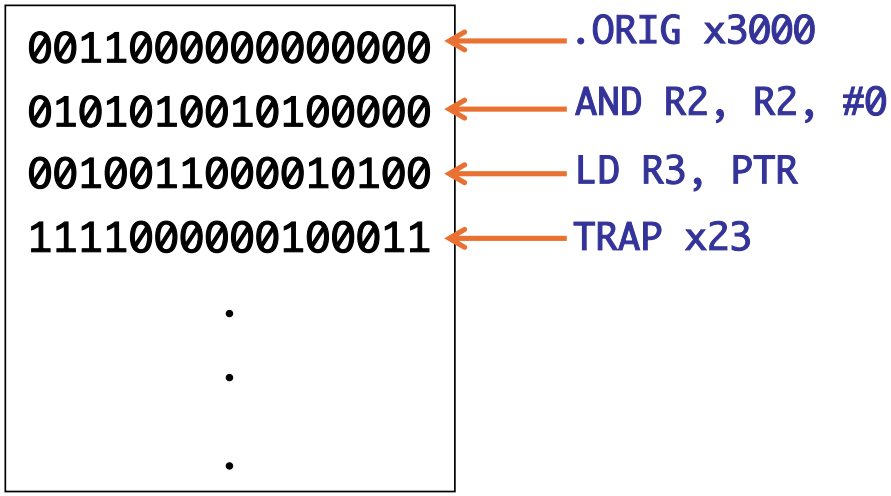
LC-3 Assembler

- The "assamble" generates several different output files



Object File Format

- LC-3 object file contains
  - Starting address (location where program must be loaded), followed by...
  - Machine instructions
- Example
  - Beginning of “count character” object file looks like this:



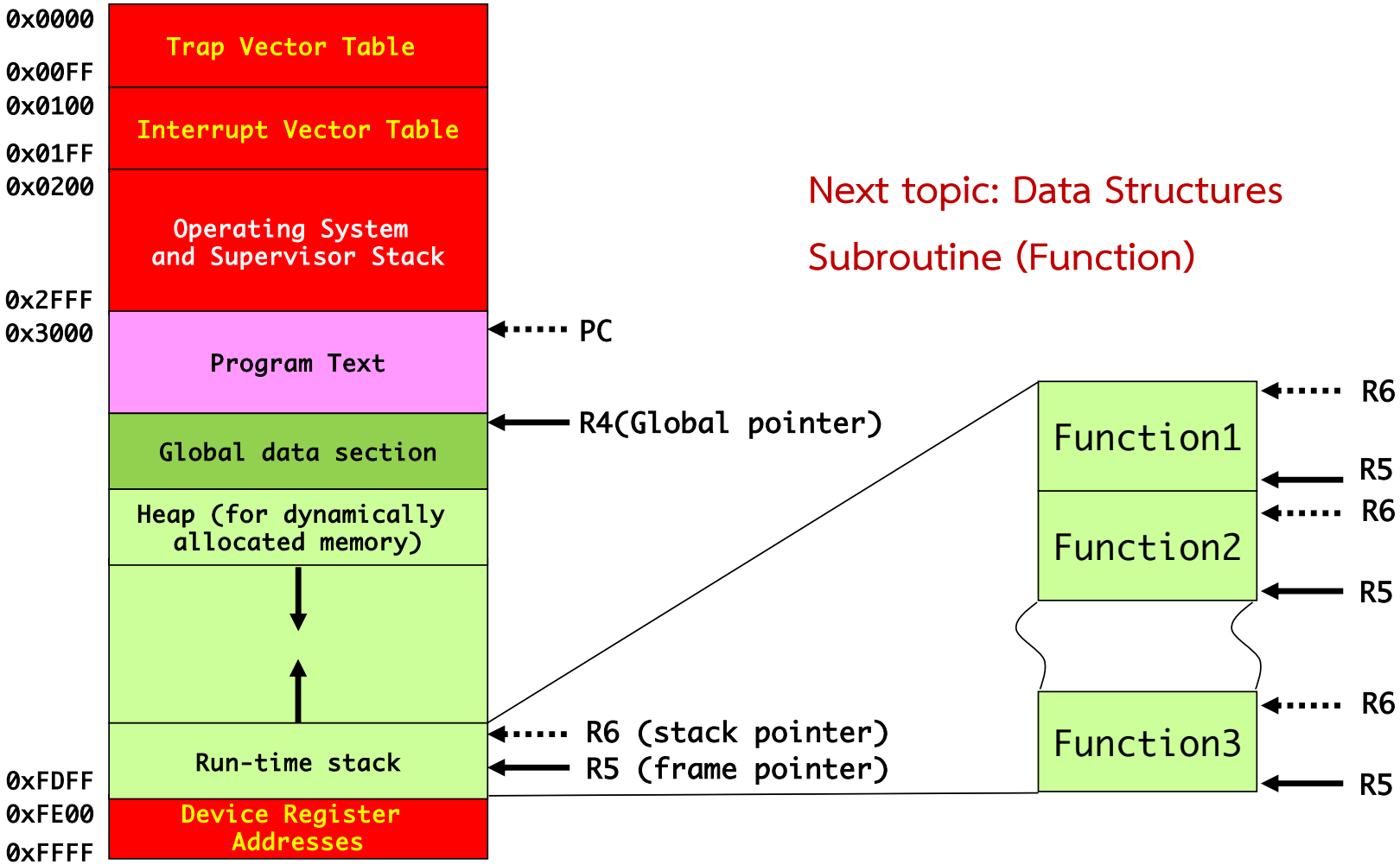
More than One Object File

- An object file is not necessarily a complete program.
  - System-provided library routines
  - Code blocks written by multiple developers
- For LC-3, can load multiple object files into memory, then start executing at a desired address.
  - System routines, such as keyboard input, are loaded automatically
    - Loaded into "system memory" below x1000
    - By convention, user code should be loaded between x3000 and xCFFF
  - Each object file includes a starting address
  - Be careful not to load overlapping object files

Linking and Loading

- *Loading* is the process of **copying** an executable image **into memory**
- *Linking* is the process of **resolving symbols** between independent object files
  - Suppose we define a symbol in one module, and want to use it in another
  - Some notation, such as **.EXTERNAL**, is used to tell assembler that **a symbol is defined in another module**
  - Linker will search symbol tables of other modules to resolve symbols and complete code generation before loading
  - **More detail in subroutine**

Memory Map of the LC-3



Next topic: Data Structures

Subroutine (Function)

Homework

- 7.1, 7.2, 7.3, 7.4, 7.5,
- 7.12, 7.19, 7.20, 7.31